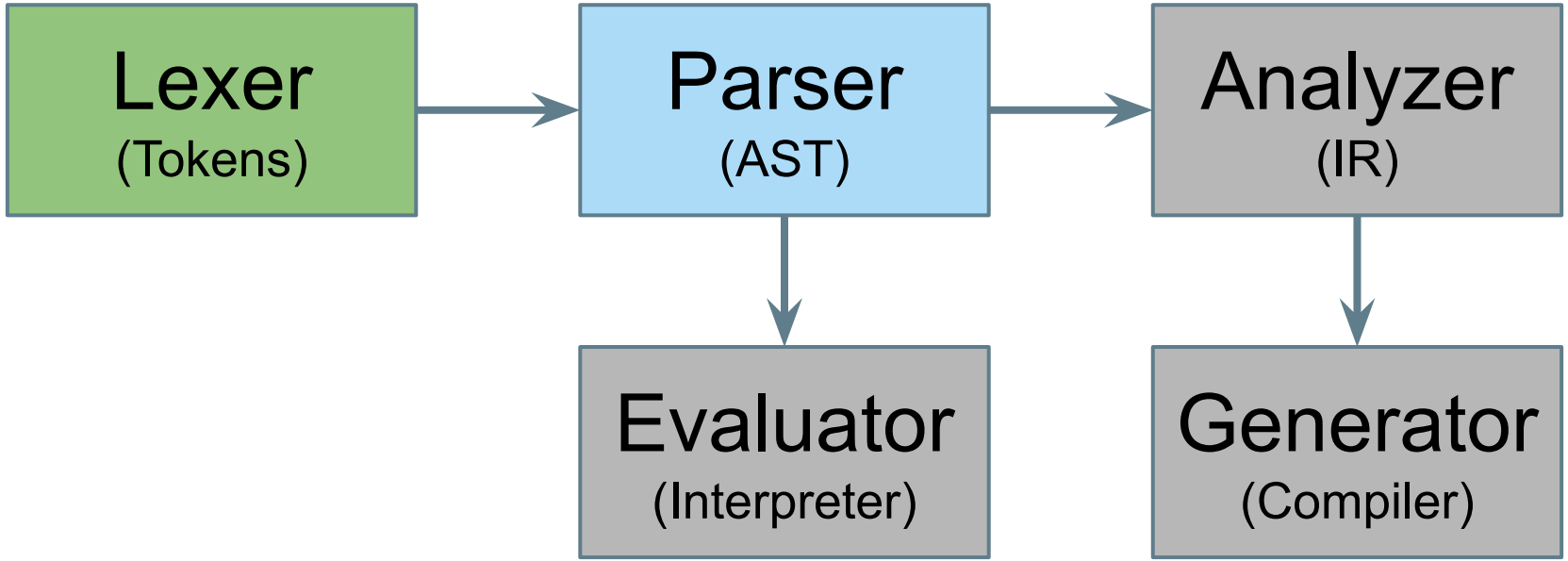
Module 3: Parsing

Parser Introduction

One down, four to go!

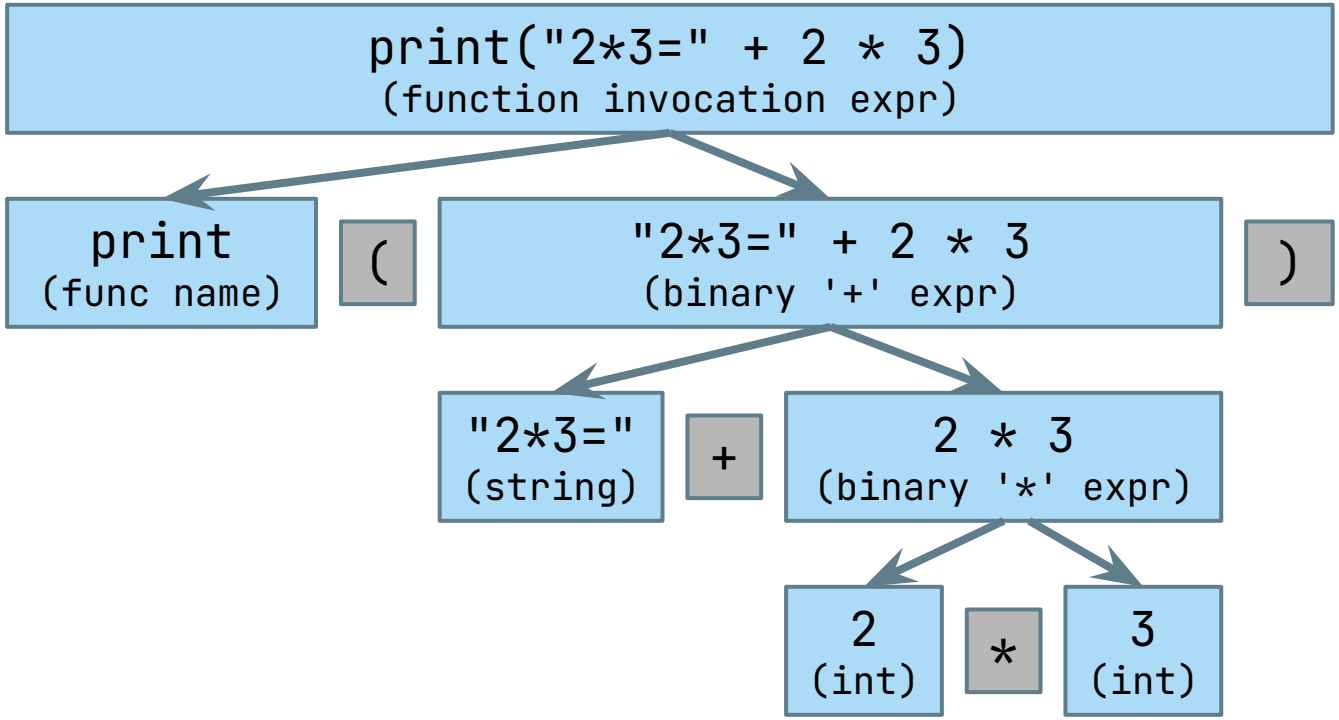
The Big Idea (!)


Lexer
(Tokens)
Parser
(AST)
Analyzer
(IR)
Evaluator
(Interpreter)
Generator
(Compiler)

1.

Parsing

I ran out of time here :(



Parser Goal: Structured Input (AST)

Parsing

Definitions

Parsing is the process of recognizing a language and determining structure.

- The structure of input encodes meaning and evaluation behavior.
- This structure is often a tree as most language are recursive.
  - expressions contain expressions!
- Handling syntax errors (where input is unclear) is also important!

Parsing Strategy

The "optimal" parsing strategy is affected by both the grammar and parsing goals.

- Grammar style/complexity:
  - Any CFG can be parsed in O(n^3) (cubic).
  - For subsets, parsing can be O(n) (linear!).
- Parsing goals (and tradeoffs):
  - Automatic parser generators exist, but can be messy and have worse error handling.
  - Parsing is a minor, relatively simple piece.
- We will use handwritten **recursive descent**.

Utilize Crafting Interpreters!

Representing Code (Grammars & Trees)

Parsing Expressions (Parsing/Precedence)

Syntax vs Semantics

Syntax

Syntax describes the textual structure of a language (in effect, the grammar).

- A variable declaration looks like:
  - `LET identifier = expr;`
- Good syntax helps quickly identify meaning based on the text alone.
  - Is `name = expr;` a variable declaration or assignment?

Semantics

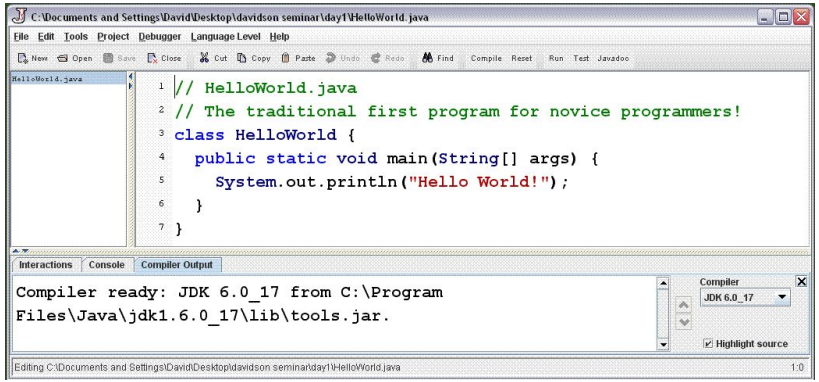
Semantics describe the actual meaning of code independent from how it is written.

- A variable declaration actually means:
  - Define “identifier” to have the value “expr”.
- Semantics cover all of the critical behavior:
  - What restrictions beyond syntax apply?
    - what if “identifier” is already defined?
  - What is the actual runtime behavior?
    - evaluate “expr”, assign to “identifier”.
- A language’s “power” comes from semantics.

Syntax “Categories”

Categories

- Structural code: Code that defines structure but isn’t really evaluated.
- Executional code: Code that is evaluated at runtime.
  - Statements: Code that performs a side effect for computation.
  - Expressions: Code that evaluates to a value (maybe side effect).



2.

Abstract Syntax Trees

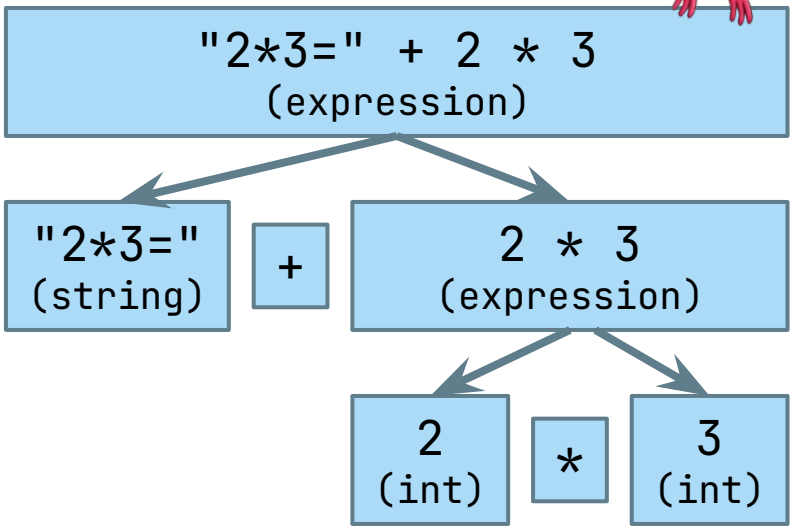
The bread and butter of all things programming language.

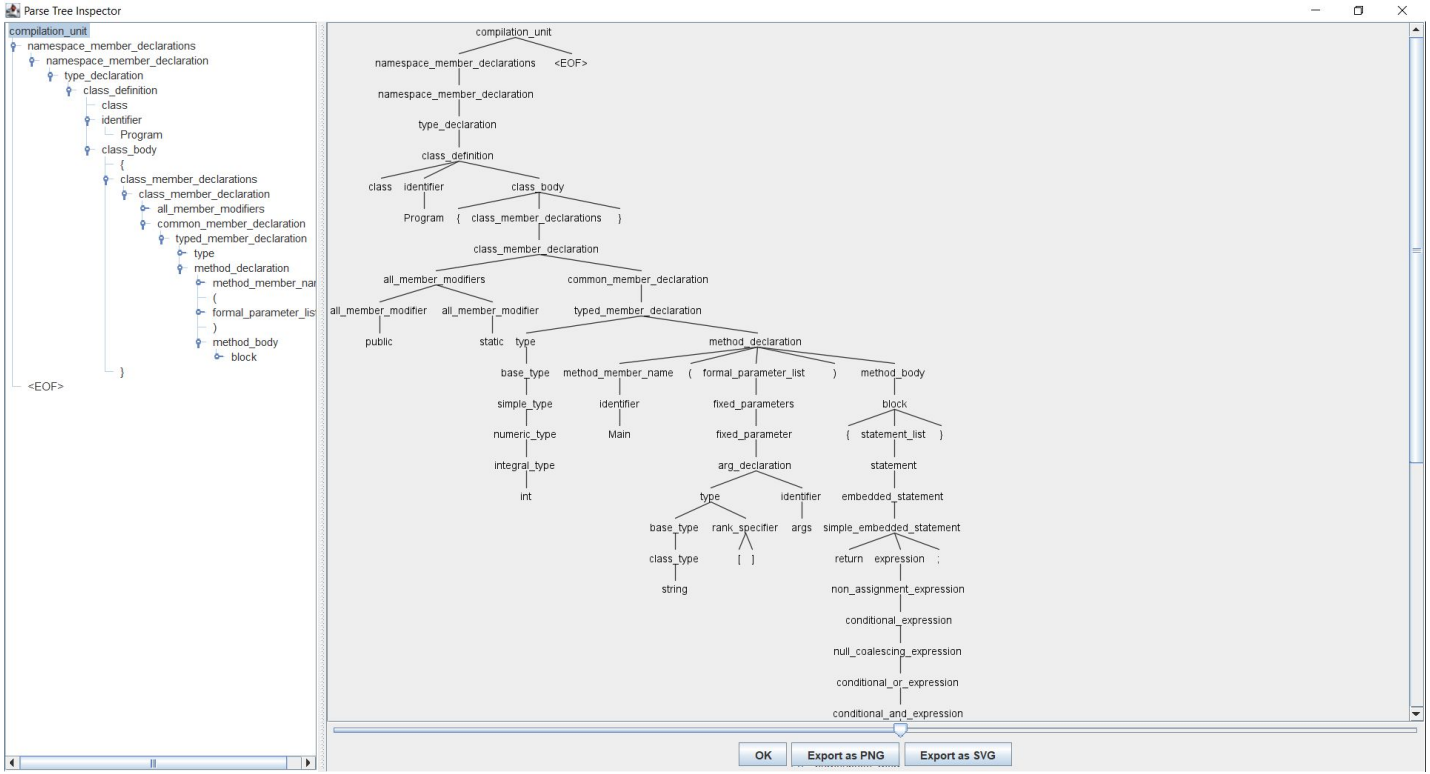
Parse Trees

Definitions

A **parse tree** shows the structure of grammar rules used to derive a string.

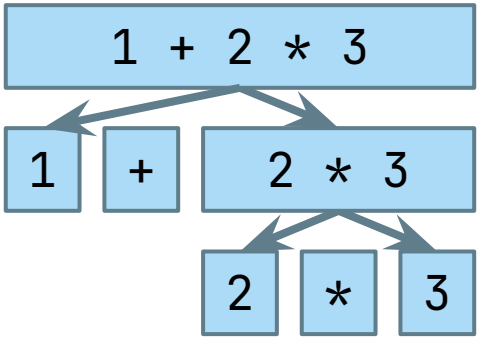
- An **ambiguous** grammar allows multiple valid parse trees.
  - This can leave the validity (and intent) of the language unclear!
- For programming languages, the parse tree correlates to the AST.

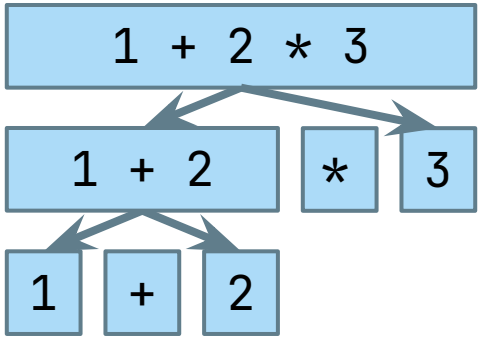




“Complete” Parse Trees

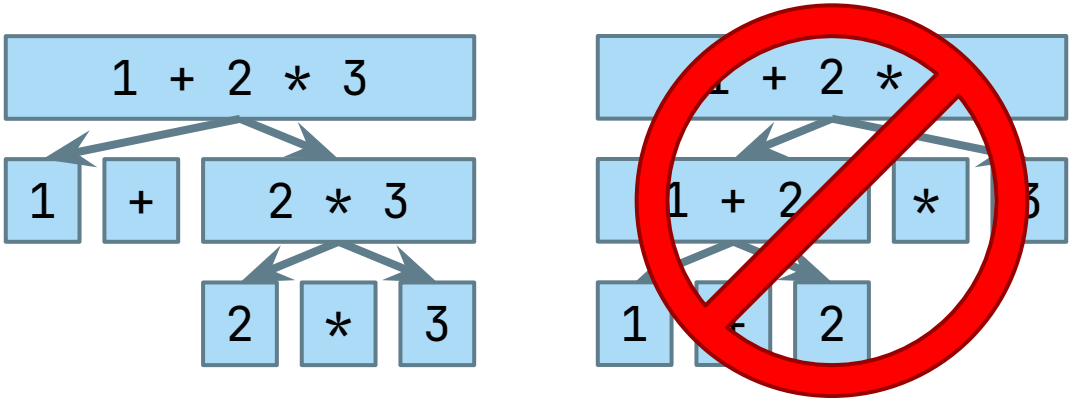
```
expression ::= [0-9]+
expression ::= expression [+-*/] expression
```





Parse Tree Ambiguity - Precedence

```
term ::= factor ([+-] factor)*
factor ::= literal ([*/] literal)*
literal ::= [0-9]+
```



Parse Tree Ambiguity - Resolution via Grammar

The *style* of Grammar matters!

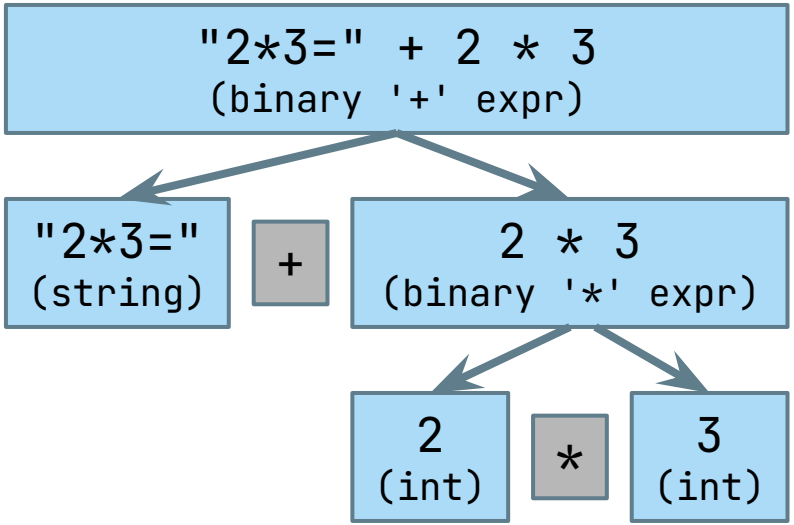
Parsing requires more than just accept/reject.
(our grammar is written for recursive descent)

Abstract Syntax Trees

Definitions

An **abstract syntax tree (AST)** encodes the structure of a language.

- Unlike a **parse tree**, the AST is separate from the grammar rules.
  - Pick the best (simplest) form.
- In practice, this means:
  - No unnecessary symbols (e.g. `;`)
  - Reduction of rules/nesting
    - Binary operators are the same!



```java
//Similar, but NOT our language!

sealed interface Expr {

  record Int(
    int value
  ) implements Expr {}

  record Binary(
    String operator,
    Expr left,
    Expr right
  ) implements Expr {}

}
```

```
"1" → Int(1)
"1+2" → Binary("+", Int(1), Int(2))
"1+2*3" →
  Binary("+",
    Int(1),
    Binary("*", Int(2), Int(3)))
"1*2+3" →
  Binary("+",
    Binary("*", Int(1), Int(2)),
    Int(3))
"1*(2+3)" →
  Binary("*",
    Int(1),
    Binary("+", Int(2), Int(3)))
//question: what happened to (...)?
//answer: it isn't needed in the AST
```

Example AST - Operator Precedence

```java
sealed interface Expr {
  record Literal(
    @Nullable Object value
  ) implements Expr {}
  record Variable(
    String name
  ) implements Expr {}
  record Property(
    Expr receiver,
    String name
  ) implements Expr {}
  record Function(
    String name,
    List<Expr> arguments
  ) implements Expr {}
}
```

```
"pow(array.length, 2.0)" →
  Function(
    name="pow",
    arguments=List.of(
      Property(
        receiver=Variable(
          name="array"
        ),
        name="length"
      ),
      Literal(
        value=new BigDecimal("2.0")
      )
    ),
  )
```

Complex Example AST

Module 3: Parsing

Parser Introduction

One down, four to go!

Slides Carnival

Free templates for all your presentation needs

For PowerPoint and Google Slides

100% free for personal or commercial use

Ready to use, professional and customizable

Blow your audience away with attractive visuals